REPORT

Analysis time complexity and handshaking logic with random vertices

Submission to : AASR

Submission by : Group – 8 (Matrix Manipulators)

Course title : Discrete Mathematics

Department: Department of Computer Science & Engineering

Group-8 (Matrix Manipulators)

1.Zahedul Islam (2024-3-60-561)

2.Nabila Anjum Diya (2024-3-60-554)

3.Tasmia Adri (2024-3-60-558)

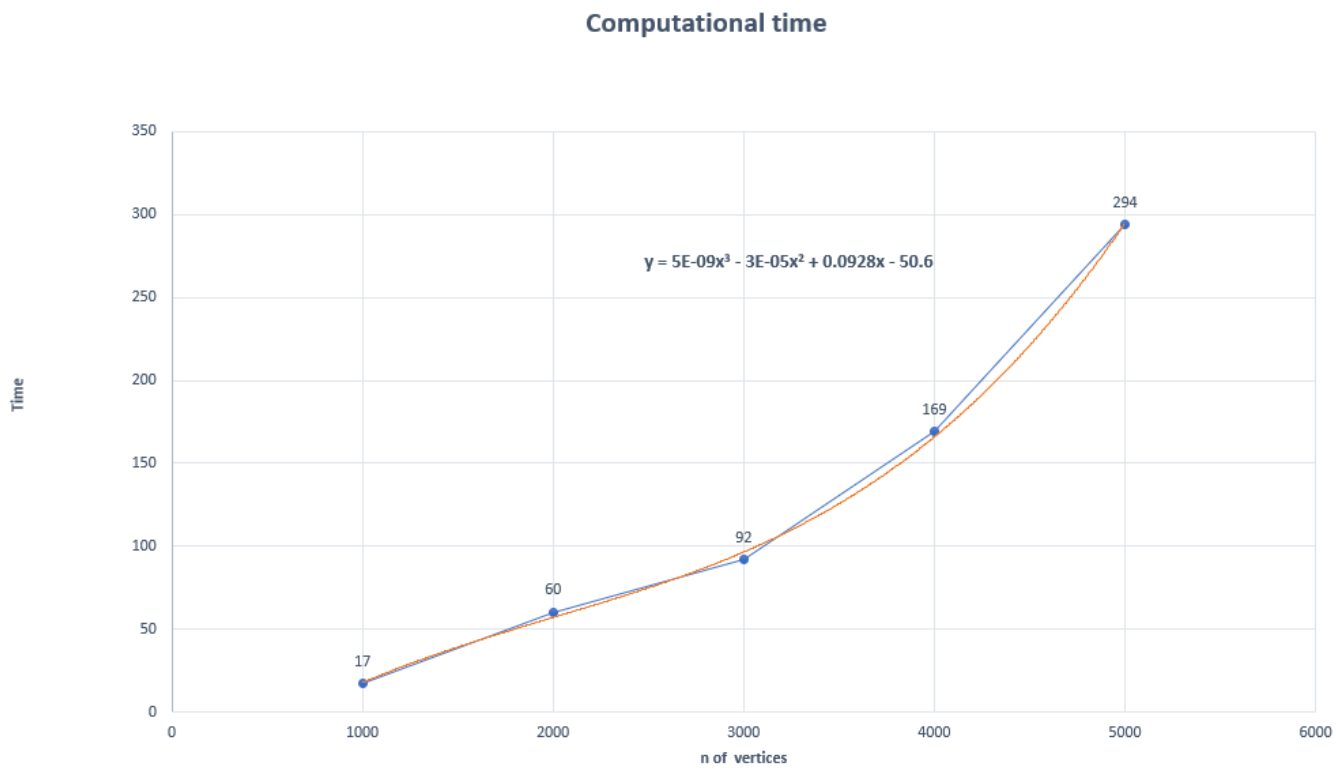
1.Using C program, randomly generate an undirected graph represented by adjacency matrix with random vertices.

The code is designed to randomly generate a matrix for various values of n, including 1000, 2000, 3000, 4000 and 5000. The output includes the total summation of degrees and edges, along with the computational time in milliseconds and checks if the handshaking Theorem holds or not.

```
For n = 1000:
sum_degrees = 499668
num_edges = 499668
Handshaking Logic holds.
Time = 17.000000:

For n = 2000:
sum_degrees = 1998900
num_edges = 1998900
Handshaking Logic holds.
Time = 60.000000:

For n = 3000:
sum_degrees = 4498490
num_edges = 4498490
Handshaking Logic holds.
Time = 92.000000:

For n = 4000:
sum_degrees = 7998024
num_edges = 7998024
Handshaking Logic holds.
Time = 169.000000:

For n = 5000:
sum_degrees = 12497316
num_edges = 12497316
Handshaking Logic holds.
Time = 294.000000:
```

2. Using Microsoft Excel Worksheet, draw a line graph showing computational time vs. n. in the line graph, select add Trendline. In the Trendline options, select Polynomial. Select the order value that mostly matches your graph. Then select Display Equation on chart. From the displayed equation , determine the time complexity of your program as a function of n.

Here, the graph illustrates the relationship between computational time and the number of vertices, resulting in the following polynomial equation.



Theoretically determine the computational time complexity of your program as a function of n and compare it with the empirical time complexity of your program as a function of n .

Polynomial equation

The time complexity

y = 5E-09n^3 – 3E-05n^2 + 0.0928n – 50.6

1.The given function ***y = 5E-09n^3 - 3E-05n^2 + 0.0928n - 50.6*** with Big-O notation, we need to focus on the term with the highest growth rate as **n** increases.

2.The term with the highest degree ***5E-09n^3*** dominates the behavior of the function as **n** grows large. The highest degree is ***n^3.***

3.In Big-O notation, we can ignore constant ***50.6*** and lower degree terms ***3E-05n^2 & 0.0928n*** because they become insignificant compared to the highest degree term as **n** increases.

4.Now, We can tell the time complexity of ***y = 5E-09n^3 - 3E-05n^2 + 0.0928n - 50.6*** as a function of **n** is ***O(n^3).***

Theoretical Time Complexity

To analyze the theoretical time complexity of the given program

1. **Graph Initialization:** Initializing an n x n adjacency matrix.
   Time complexity: *O(n^2)*
2. **Degree initialization:** Setting each element in the degree array to 0.
   Time complexity: *O(n)*
3. **Random Graph Generation**: Iterating over vertex pairs and updating the matrix and degrees.
   Time Complexity: *O(n^2)*
4. **Degree Summation and Handshaking Logic Verification**: Summing degrees
   And checking the handshaking logic.
   Time complexity: *O(n)*

Compare the theoretical and empirical Time complexity

1. Theoretical complexity, Summing the degrees and counting edges runs in ***O(n^2)*** time.
2. The empirical complexity from the Excel Graph also follow ***O(n^3).***

Conclusion : After comparison we can see both result are not same.